SOLAR ENERGY GENERATION AND PRICE GENERATION

-DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Course Code and Name: UCS310 –
Database Management Systems

B.Tech (2nd Year)
Computer Engineering - Thapar Institute of Engineering and Technology

Submitted By:
Tarun Mahindra - 1024030177
Parth Puri - 1024030178

Submitted To:
Dr. Simran

Academic Year:
Jan-May, 2026

Table of Contents

1. Introduction

The rapid global adoption of rooftop solar panels has fundamentally disrupted traditional energy supply chains. In India alone, the Ministry of New and Renewable Energy reported over 11 GW of rooftop solar capacity as of 2024, with thousands of households and commercial establishments generating surplus electricity daily. However, the absence of a structured peer-to-peer mechanism means that most of this surplus energy is either wasted or fed back into the grid at unfavourable rates, with no direct economic benefit to the producer.

This project — the **Solar Energy Peer-to-Peer Trading and Dynamic Pricing System** — addresses this gap by implementing a full-stack, database-driven platform that allows energy producers (PRODUCER role) and energy consumers (CONSUMER role) to trade surplus solar energy directly, with pricing determined dynamically by supply-demand conditions encoded in Oracle PL/SQL business logic.

At its core the system is a relational database application built on Oracle Database with PL/SQL stored packages acting as the primary business logic engine. A lightweight Node.js / Express REST API bridges the database layer with an HTML/CSS/JavaScript frontend, giving the project a realistic three-tier architecture comparable to production energy trading applications.

1.1 Problem Statement & Motivation

Traditional energy grids are designed as one-directional systems: electricity flows from large power plants to end consumers. The rise of distributed renewable generation breaks this paradigm, creating millions of micro-producers who need a marketplace to sell their surplus. Key pain points include:

- No transparent mechanism for peer-to-peer energy trading at the local grid level.
- Static tariff structures that do not reflect real-time supply and demand.
- Manual, paper-based record-keeping that is error-prone and non-auditable.
- Lack of wallet / payment tracking integrated with the energy exchange.
- No admin oversight layer to approve participants and maintain system integrity.

1.2 Objectives

1. Design a fully normalised (3NF) relational schema	Covering users, wallets, energy listings, transactions, meter readings and admin logs.
2. Implement a PL/SQL business logic layer	Using Oracle packages (PKG_USER, PKG_TRADING, PKG_REPORT) to encapsulate all trading rules.
3. Build a RESTful API gateway	Node.js / Express translates HTTP requests into PL/SQL stored-procedure calls.
4. Develop a role-based frontend	Separate views for PRODUCER, CONSUMER and ADMIN roles.
5. Enforce data integrity automatically	Through CHECK constraints, FOREIGN KEY constraints and BEFORE/AFTER triggers.
6. Enable real-time wallet management	Atomic debit/credit during each energy transaction with balance validation.

7. Support admin governance	Pending-user approval workflow and an audit log for all system events.
8. Provide analytical reporting	Views and queries that expose trading volume, wallet balances and transaction history.

1.3 Project Scope

The project is implemented as an "Iteration 2" full-stack application, extending a first-iteration pure-SQL prototype into a working web application. The scope covers:

- User registration, login and admin-controlled approval workflow.
- Automatic wallet creation (seeded with ■1,000 credit) for every new user via trigger.
- Energy listing creation by approved PRODUCERs with unit and price specifications.
- Marketplace browsing — consumers view open listings sorted by price.
- Atomic buy_energy transaction: wallet debit (buyer), wallet credit (seller), listing unit decrement, transaction record creation — all in one PL/SQL procedure.
- Meter reading submission with trigger-computed excess units.
- Admin dashboard: pending-user approval queue and full admin audit log.
- Demo mode: frontend and Node backend run without a live Oracle instance using an in-memory mock data store.

Out of scope: Real smart-meter hardware integration, inter-grid settlement with DISCOMs, mobile applications, and cryptocurrency payment rails.

1.4 Technology Stack

Layer	Technology	Version / Details
Database	Oracle Database	Oracle 19c / Oracle XE / Free
PL/SQL Packages	Oracle PL/SQL	Packages, triggers, ref cursors
API Gateway	Node.js + Express	Node 20+, Express 4.x, ESM modules
Oracle Driver	node-oracledb	v6.7.1
Frontend	HTML5 / CSS3 / Vanilla JS	No external frameworks
Demo Mode	In-memory mock DB	demoGateway.js
Version Control	Git	Monorepo layout
Report	ReportLab (Python)	PDF generation

2. System Architecture & Design

2.1 High-Level Architecture

The system follows a classic **three-tier client–server architecture**: a browser-based presentation tier, a Node.js application/API tier, and an Oracle database tier. All core business logic resides in the Oracle database layer through PL/SQL packages, ensuring that the application tier remains thin and the database layer remains the authoritative source of truth.

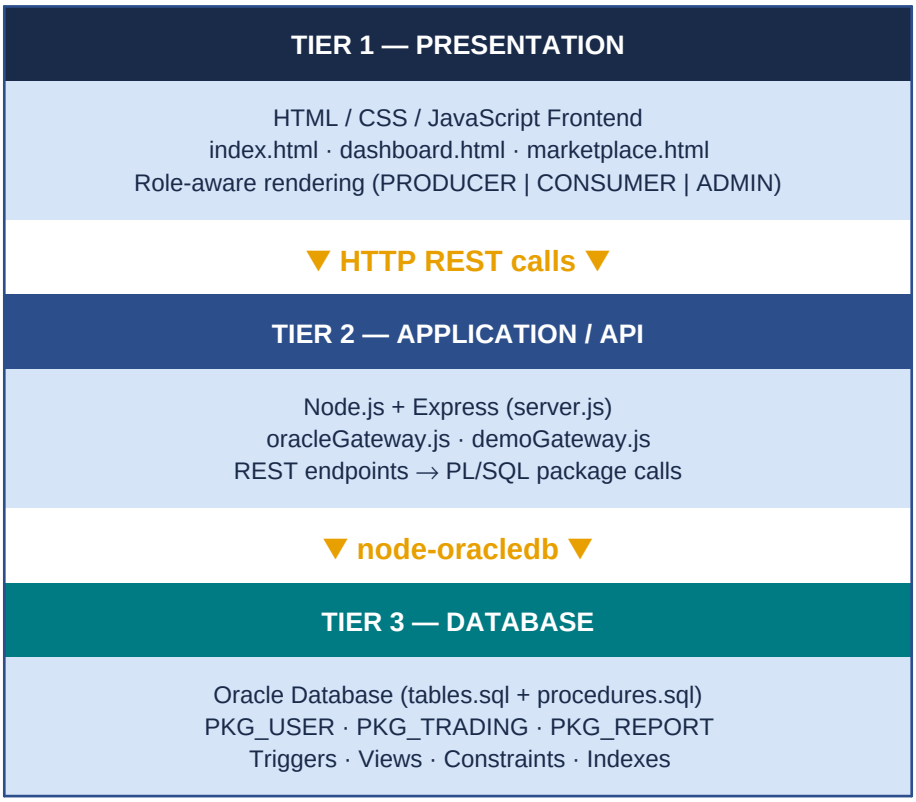


Figure 2.1 — Three-Tier System Architecture

2.2 Three-Tier Application Design

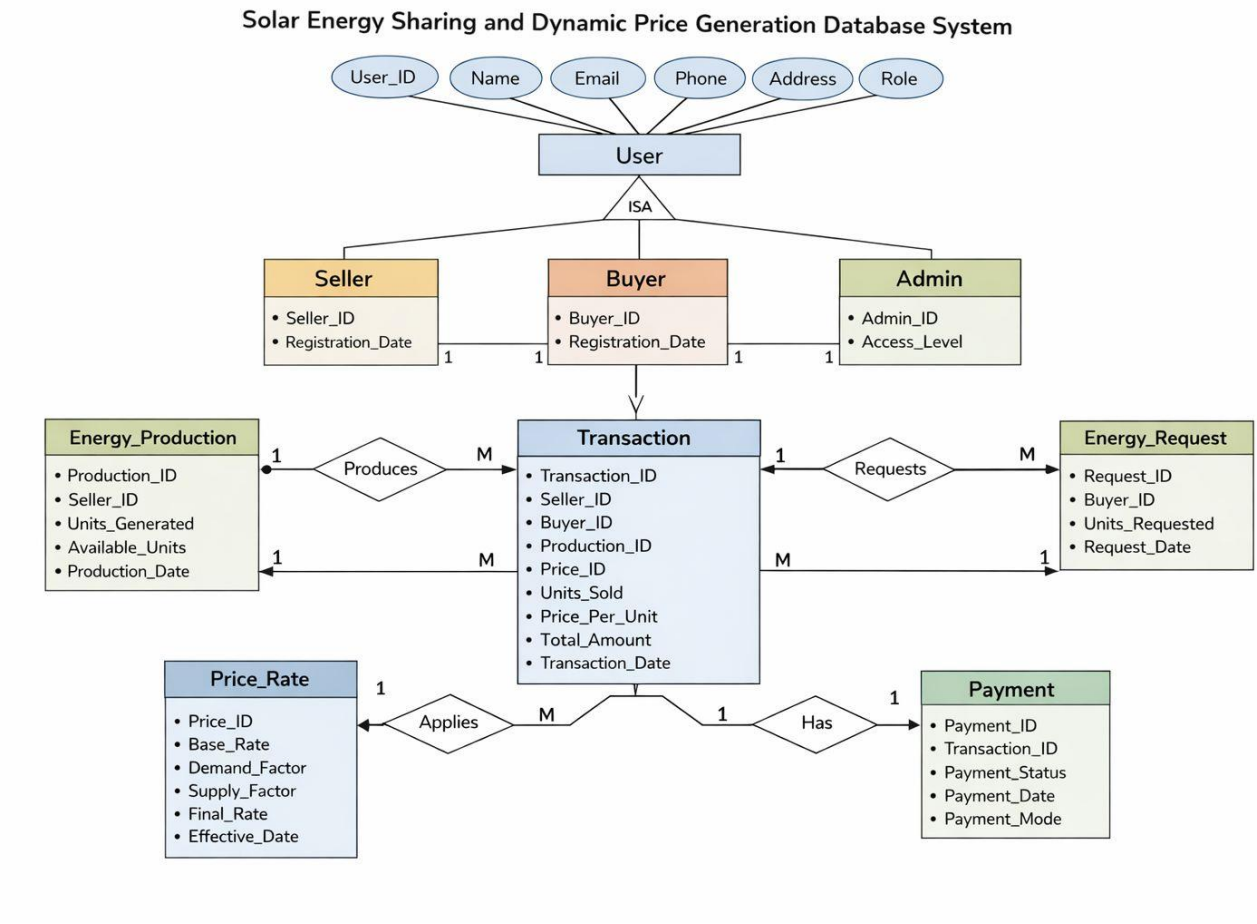
Each tier has clearly defined responsibilities, ensuring loose coupling and high cohesion:

Tier	Component	Responsibility
Presentation	index.html, dashboard.html, marketplace.html	User interaction, role-based UI rendering, REST API consumption
Application	server.js, oracleGateway.js, demoGateway.js	HTTP routing, session handling, Oracle connection pool, procedure invocation
Database	tables.sql, procedures.sql, triggers, views	Data storage, business rule enforcement, transaction atomicity, audit logging

2.3 Entity-Relationship Model

The ER model captures the real-world entities, their attributes, and the relationships between them. The system evolved from the initial 9-table conceptual design to a streamlined 6-table implementation that

- ER DIAGRAM:


Solar Energy Sharing and Dynamic Price Generation Database System
User_ID
Name
Email
Phone
Address
Role
User
ISA
Seller
• Seller_ID
• Registration_Date
Buyer
• Buyer_ID
• Registration_Date
Admin
• Admin_ID
• Access_Level
1
1
1
1
Energy_Production
• Production_ID
• Seller_ID
• Units_Generated
• Available_Units
• Production_Date
1
Produces
M
1
M
Transaction
• Transaction_ID
• Seller_ID
• Buyer_ID
• Production_ID
• Price_ID
• Units_Sold
• Price_Per_Unit
• Total_Amount
• Transaction_Date
1
Requests
M
M
1
Energy_Request
• Request_ID
• Buyer_ID
• Units_Requested
• Request_Date
Price_Rate
• Price_ID
• Base_Rate
• Demand_Factor
• Supply_Factor
• Final_Rate
• Effective_Date
1
Applies
M
1
Has
1
Payment
• Payment_ID
• Transaction_ID
• Payment_Status
• Payment_Date
• Payment_Mode

integrates wallet management and meter reading directly into the core schema.

Entity	Primary Key	Key Attributes	Role in System
USERS	user_id	name, email, password, role, zone_name, status	Central identity entity; parent to WALLETS, ENERGY_LISTINGS, ENERGY_TRANSACTIONS, METER_READINGS
WALLETS	wallet_id	user_id (FK), balance	Financial ledger per user; auto-created by trigger on INSERT to USERS
ENERGY_LISTINGS	listing_id	producer_id (FK), units_available, price_per_unit, energy_type, status	Published energy offers by approved PRODUCERs
ENERGY_TRANSACTIONS	transaction_id	buyer_id (FK), seller_id (FK), listing_id (FK), units_bought, total_amount, transaction_date	Immutable record of every energy purchase
METER_READINGS	reading_id	user_id (FK), units_generated, units_consumed, excess_units, reading_time	Tracks physical generation & consumption; excess computed by trigger
ADMIN_LOGS	log_id	action, log_time	Append-only audit trail for all system events

2.4 Relational Schema Design

The relational schema below shows every table with its column definitions, data types, and constraints. Oracle's IDENTITY columns are used for auto-incrementing primary keys.

Table: USERS

Column	Data Type	Constraints / Notes
`user_id`	NUMBER	PK, IDENTITY
`name`	VARCHAR2(80)	NOT NULL
`email`	VARCHAR2(100)	UNIQUE NOT NULL
`password`	VARCHAR2(50)	NOT NULL
`role`	VARCHAR2(20)	CHECK IN ('PRODUCER','CONSUMER','ADMIN')
`zone_name`	VARCHAR2(50)	DEFAULT 'Local Grid'
`status`	VARCHAR2(20)	DEFAULT 'PENDING', CHECK IN ('PENDING','APPROVED','REJECTED')
`created_at`	DATE	DEFAULT SYSDATE

Table: WALLETS

Column	Data Type	Constraints / Notes

wallet_id	NUMBER	PK, IDENTITY
user_id	NUMBER	FK → USERS, UNIQUE NOT NULL
balance	NUMBER(10,2)	DEFAULT 0, CHECK >= 0

Table: ENERGY_LISTINGS

Column	Data Type	Constraints / Notes
listing_id	NUMBER	PK, IDENTITY
producer_id	NUMBER	FK → USERS NOT NULL
units_available	NUMBER(8,2)	CHECK >= 0
price_per_unit	NUMBER(8,2)	CHECK > 0
energy_type	VARCHAR2(30)	DEFAULT 'SOLAR'
status	VARCHAR2(20)	DEFAULT 'OPEN', CHECK IN ('OPEN','SOLD','CLOSED')
created_at	DATE	DEFAULT SYSDATE

Table: ENERGY_TRANSACTIONS

Column	Data Type	Constraints / Notes
transaction_id	NUMBER	PK, IDENTITY
buyer_id	NUMBER	FK → USERS NOT NULL
seller_id	NUMBER	FK → USERS NOT NULL
listing_id	NUMBER	FK → ENERGY_LISTINGS NOT NULL
units_bought	NUMBER(8,2)	CHECK > 0
total_amount	NUMBER(10,2)	CHECK > 0
transaction_date	DATE	DEFAULT SYSDATE

Table: METER_READINGS

Column	Data Type	Constraints / Notes
reading_id	NUMBER	PK, IDENTITY
user_id	NUMBER	FK → USERS NOT NULL
units_generated	NUMBER(8,2)	DEFAULT 0
units_consumed	NUMBER(8,2)	DEFAULT 0
excess_units	NUMBER(8,2)	DEFAULT 0 (set by trigger)
reading_time	DATE	DEFAULT SYSDATE

Table: ADMIN_LOGS

Column	Data Type	Constraints / Notes
log_id	NUMBER	PK, IDENTITY

`action`	VARCHAR2(200)	NOT NULL
`log_time`	DATE	DEFAULT SYSDATE

2.5 Normalization Analysis

The schema is designed to satisfy Third Normal Form (3NF), eliminating all data redundancy and ensuring update anomalies cannot occur.

Normal Form	Requirement	How it is Satisfied
1NF	All attributes are atomic; no repeating groups.	Every column holds a single, indivisible value. Roles and statuses use lookup-style CHECK constraints rather than multi-valued fields.
2NF	No partial dependencies on a composite PK.	All tables use single-column surrogate primary keys (IDENTITY), so partial dependencies are structurally impossible.
3NF	No transitive dependencies.	Derived values (excess_units, total_amount) are computed at write time by triggers/procedures and stored for performance, with no hidden transitive chain. Producer name is not stored in ENERGY_LISTINGS; it is obtained via JOIN to USERS.

Specific normalization decisions: WALLETS is a separate table (not columns inside USERS) to separate financial concerns and enforce the 1:1 relationship via a UNIQUE FK. ADMIN_LOGS is append-only with no FK to USERS, allowing logs to survive user deletions.

2.6 Referential Integrity & Constraints

Constraint Type	Table(s)	Constraint Name	Rule Enforced
PRIMARY KEY	All tables	Auto-named	Unique row identity
FOREIGN KEY	WALLETS	fk_wallet_user	wallet.user_id → users.user_id
FOREIGN KEY	ENERGY_LISTINGS	fk_listing_producer	listing.producer_id → users.user_id
FOREIGN KEY	ENERGY_TRANSACTIONS	fk_tx_buyer / fk_tx_seller / fk_tx_listing	All three FKs into USERS and ENERGY_LISTINGS
FOREIGN KEY	METER_READINGS	fk_meter_user	reading.user_id → users.user_id
UNIQUE	USERS	On email	No duplicate accounts
UNIQUE	WALLETS	On user_id	One wallet per user
CHECK	USERS	chk_user_role	role IN ('PRODUCER','CONSUMER','ADMIN')
CHECK	USERS	chk_user_status	status IN ('PENDING','APPROVED','REJECTED')

Constraint Type	Table(s)	Constraint Name	Rule Enforced
CHECK	WALLETS	chk_wallet_balance	balance >= 0
CHECK	ENERGY_LISTINGS	chk_listing_units / chk_listing_price / chk_listing_status	units >= 0, price > 0, status valid
CHECK	ENERGY_TRANSACTIONS	chk_tx_units / chk_tx_total	units > 0, total > 0
NOT NULL	Multiple	Various	Critical fields cannot be null

2.7 Indexes & Views

Three B-tree indexes are created to accelerate the most common query patterns:

Index Name	Table	Column	Query Pattern Accelerated
idx_listing_status	ENERGY_LISTINGS	status	Marketplace: WHERE status = 'OPEN'
idx_tx_buyer	ENERGY_TRANSACTIONS	buyer_id	Transaction history by buyer
idx_tx_seller	ENERGY_TRANSACTIONS	seller_id	Revenue reports by seller

Database Views:

View Name	Definition	Purpose
VW_AVAILABLE_LISTINGS	JOIN ENERGY_LISTINGS + USERS WHERE status='OPEN' AND units > 0 AND user.status='APPROVED'	Powers the consumer marketplace; hides listings from unapproved producers
VW_TRANSACTION_DETAILS	JOIN ENERGY_TRANSACTIONS + USERS (buyer alias + seller alias)	Human-readable transaction history with buyer/seller names

3. Database Implementation

3.1 DDL — Table Definitions (tables.sql)

The following is the complete, annotated DDL for the six core tables. Oracle's **GENERATED BY DEFAULT ON NULL AS IDENTITY** syntax is used for primary keys, providing auto-increment semantics compatible with INSERT statements that omit the PK column.

```
-- ■■ USERS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE USERS (
user_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
name VARCHAR2(80) NOT NULL,
email VARCHAR2(100) UNIQUE NOT NULL,
password VARCHAR2(50) NOT NULL,
role VARCHAR2(20) NOT NULL,
zone_name VARCHAR2(50) DEFAULT 'Local Grid',
status VARCHAR2(20) DEFAULT 'PENDING' NOT NULL,
created_at DATE DEFAULT SYSDATE,
CONSTRAINT chk_user_role CHECK (role IN ('PRODUCER','CONSUMER','ADMIN')),
CONSTRAINT chk_user_status CHECK (status IN ('PENDING','APPROVED','REJECTED'))
);

-- ■■ WALLETS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE WALLETS (
wallet_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
user_id NUMBER UNIQUE NOT NULL,
balance NUMBER(10,2) DEFAULT 0 NOT NULL,
CONSTRAINT fk_wallet_user FOREIGN KEY (user_id) REFERENCES USERS(user_id),
CONSTRAINT chk_wallet_balance CHECK (balance >= 0)
);

-- ■■ ENERGY_LISTINGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE ENERGY_LISTINGS (
listing_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
producer_id NUMBER NOT NULL,
units_available NUMBER(8,2) NOT NULL,
price_per_unit NUMBER(8,2) NOT NULL,
energy_type VARCHAR2(30) DEFAULT 'SOLAR',
status VARCHAR2(20) DEFAULT 'OPEN' NOT NULL,
created_at DATE DEFAULT SYSDATE,
CONSTRAINT fk_listing_producer FOREIGN KEY (producer_id) REFERENCES USERS(user_id),
CONSTRAINT chk_listing_units CHECK (units_available >= 0),
CONSTRAINT chk_listing_price CHECK (price_per_unit > 0),
CONSTRAINT chk_listing_status CHECK (status IN ('OPEN','SOLD','CLOSED'))
);
-- ■■ ENERGY_TRANSACTIONS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE ENERGY_TRANSACTIONS (
transaction_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
buyer_id NUMBER NOT NULL,
seller_id NUMBER NOT NULL,
listing_id NUMBER NOT NULL,
units_bought NUMBER(8,2) NOT NULL,
total_amount NUMBER(10,2) NOT NULL,
transaction_date DATE DEFAULT SYSDATE,
CONSTRAINT fk_tx_buyer FOREIGN KEY (buyer_id) REFERENCES USERS(user_id),
```

```
CONSTRAINT fk_tx_seller FOREIGN KEY (seller_id) REFERENCES USERS(user_id),
CONSTRAINT fk_tx_listing FOREIGN KEY (listing_id) REFERENCES ENERGY_LISTINGS(listing_id),
CONSTRAINT chk_tx_units CHECK (units_bought > 0),
CONSTRAINT chk_tx_total CHECK (total_amount > 0)
);

-- ■■ METER_READINGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE METER_READINGS (
reading_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
user_id NUMBER NOT NULL,
units_generated NUMBER(8,2) DEFAULT 0,
units_consumed NUMBER(8,2) DEFAULT 0,
excess_units NUMBER(8,2) DEFAULT 0, -- auto-computed by trigger
reading_time DATE DEFAULT SYSDATE,
CONSTRAINT fk_meter_user FOREIGN KEY (user_id) REFERENCES USERS(user_id)
);

-- ■■ ADMIN_LOGS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE TABLE ADMIN_LOGS (
log_id NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
action VARCHAR2(200) NOT NULL,
log_time DATE DEFAULT SYSDATE
);

-- ■■ INDEXES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
CREATE INDEX idx_listing_status ON ENERGY_LISTINGS(status);
CREATE INDEX idx_tx_buyer ON ENERGY_TRANSACTIONS(buyer_id);
CREATE INDEX idx_tx_seller ON ENERGY_TRANSACTIONS(seller_id);
```

3.2 Triggers

Two database triggers provide automatic, transparent behaviour that application code cannot bypass — ensuring data integrity at the lowest possible level.

Trigger 1: TRG_CREATE_WALLET (AFTER INSERT on USERS)

Every new user record automatically receives a wallet seeded with ■1,000 credit. This eliminates the possibility of a user existing without a wallet (which would violate the 1:1 business rule) and removes the need for application code to explicitly call a wallet-creation routine.

```
CREATE OR REPLACE TRIGGER TRG_CREATE_WALLET
AFTER INSERT ON USERS
FOR EACH ROW
BEGIN
INSERT INTO WALLETS(user_id, balance)
VALUES(:NEW.user_id, 1000);
END;
/
```

Trigger 2: TRG_METER_EXCESS (BEFORE INSERT on METER_READINGS)

When a meter reading is submitted, the trigger intercepts it before the row is written and automatically computes the excess units as max(generated – consumed, 0). Using GREATEST() ensures the value is never negative even if consumption exceeds generation (e.g., during cloudy days). This derived value is stored for fast reporting without requiring a recomputation on every read.

```
CREATE OR REPLACE TRIGGER TRG_METER_EXCESS
BEFORE INSERT ON METER_READINGS
```

```
FOR EACH ROW
BEGIN
:NEW.excess_units := GREATEST(:NEW.units_generated - :NEW.units_consumed, 0);
END;
/
```

3.3 Seed Data & Sample Dataset

The seed script (seed.sql) populates the database with three demo users and a sample listing for classroom demonstration. All demo passwords are "1234".

user_id	Name	Email	Role	Zone	Status	Wallet Balance
1	Ravi Solar House	producer@energy.com	PRODUCER	North Zone	APPROVED	■500
2	Anita Consumer	consumer@energy.com	CONSUMER	North Zone	APPROVED	■1,000
3	Admin User	admin@energy.com	ADMIN	Main Grid	APPROVED	■0

Sample Energy Listing:

listing_id	producer_id	units_available	price_per_unit	energy_type	status
1	1 (Ravi Solar House)	50	■6.50	SOLAR	OPEN

Sample Meter Reading:

reading_id	user_id	units_generated	units_consumed	excess_units (trigger)
1	1 (Ravi)	30	12	18 ← computed automatically by TRG_METER_EXCESS

4. PL/SQL Logic Layer

All business logic is encapsulated in three Oracle PL/SQL packages. This design decision means that any application tier (Node.js, Python, Java, or even SQL*Plus) can invoke the same consistent, tested procedures. The Node.js API gateway is therefore intentionally thin — it maps HTTP requests to package procedure calls and returns JSON to the frontend.

4.1 Package Overview

Package	Procedures / Functions	Responsibility
PKG_USER	register_user, login_user, approve_user	User lifecycle management: registration, authentication, admin approval
PKG_TRADING	add_listing, get_listings, buy_energy, add_meter_reading	Core energy trading engine: listing creation, marketplace query, atomic purchase
PKG_REPORT	get_wallet, get_transactions, get_pending_users, get_admin_logs	Read-only reporting: wallet balance, transaction history, pending approvals, audit log

4.2 PKG_USER — User Management

register_user Procedure

Inserts a new user into USERS, trimming and lowercasing the email for consistency, and upcasing the role. Raises *ORA-20001* if the email already exists (DUP_VAL_ON_INDEX). The TRG_CREATE_WALLET trigger fires automatically after the INSERT, creating the wallet.

```
PROCEDURE register_user(
p_name IN VARCHAR2,
p_email IN VARCHAR2,
p_password IN VARCHAR2,
p_role IN VARCHAR2,
p_zone_name IN VARCHAR2,
p_user_id OUT NUMBER
) AS
BEGIN
INSERT INTO USERS(name, email, password, role, zone_name)
VALUES(TRIM(p_name), LOWER(TRIM(p_email)), p_password, UPPER(p_role), p_zone_name)
RETURNING user_id INTO p_user_id;

INSERT INTO ADMIN_LOGS(action) VALUES('New user registered: ' || p_name);
EXCEPTION
WHEN DUP_VAL_ON_INDEX THEN
RAISE_APPLICATION_ERROR(-20001, 'Email already exists');
END register_user;
```

login_user Procedure

Opens a REF CURSOR over the USERS table filtered by email and password. The Node.js gateway fetches the cursor rows and maps them to a JSON response. Note: production systems would use hashed passwords; this project uses plaintext for classroom simplicity.

```
PROCEDURE login_user(
p_email IN VARCHAR2,
p_password IN VARCHAR2,
p_result OUT SYS_REFCURSOR
) AS
BEGIN
OPEN p_result FOR
SELECT user_id, name, email, role, zone_name, status
FROM USERS
WHERE LOWER(email) = LOWER(TRIM(p_email))
AND password = p_password;
END login_user;
```

approve_user Procedure

Admin-only operation that updates a user's status to APPROVED or REJECTED and writes an audit entry to ADMIN_LOGS.

```
PROCEDURE approve_user(p_user_id IN NUMBER, p_status IN VARCHAR2) AS
BEGIN
UPDATE USERS SET status = UPPER(p_status) WHERE user_id = p_user_id;
INSERT INTO ADMIN_LOGS(action)
VALUES('User ' || p_user_id || ' marked as ' || UPPER(p_status));
END approve_user;
```

4.3 PKG_TRADING — Core Trading Engine

add_listing Procedure

Validates that the caller is an APPROVED PRODUCER before inserting a new energy listing. Raises application errors (-20101, -20102) for invalid role or unapproved status.

```
PROCEDURE add_listing(
p_producer_id IN NUMBER, p_units IN NUMBER,
p_price IN NUMBER, p_energy_type IN VARCHAR2, p_listing_id OUT NUMBER
) AS
v_role USERS.role%TYPE;
v_status USERS.status%TYPE;
BEGIN
SELECT role, status INTO v_role, v_status FROM USERS WHERE user_id = p_producer_id;
IF v_role <> 'PRODUCER' THEN
RAISE_APPLICATION_ERROR(-20101, 'Only producers can add listings');
END IF;
IF v_status <> 'APPROVED' THEN
RAISE_APPLICATION_ERROR(-20102, 'Producer is not approved');
END IF;
INSERT INTO ENERGY_LISTINGS(producer_id, units_available, price_per_unit, energy_type)
VALUES(p_producer_id, p_units, p_price, UPPER(p_energy_type))
RETURNING listing_id INTO p_listing_id;
INSERT INTO ADMIN_LOGS(action) VALUES('Producer ' || p_producer_id || ' added a listing');
END add_listing;
```

buy_energy Procedure — Atomic Transaction

This is the most critical procedure in the system. It performs a complete, atomic energy purchase in a single database transaction, applying six sequential operations all protected by implicit Oracle transaction boundaries:

- **Step 1:** Validate that the caller is an APPROVED CONSUMER.

- **Step 2:** Lock and read the ENERGY_LISTINGS row (SELECT ... FOR UPDATE) to prevent concurrent overselling.
- **Step 3:** Confirm requested units do not exceed available units.
- **Step 4:** Lock and read the buyer's WALLETS row (FOR UPDATE) to prevent concurrent overdraft.
- **Step 5:** Confirm the buyer's wallet balance covers the total cost.
- **Step 6:** Debit the buyer's wallet and credit the seller's wallet atomically.
- **Step 7:** Decrement listing units; auto-set status to 'SOLD' if units reach zero.
- **Step 8:** Insert into ENERGY_TRANSACTIONS and ADMIN_LOGS.

```
PROCEDURE buy_energy(
p_buyer_id IN NUMBER, p_listing_id IN NUMBER,
p_units IN NUMBER, p_transaction_id OUT NUMBER
) AS
v_buyer_role USERS.role%TYPE;
v_buyer_status USERS.status%TYPE;
v_seller_id NUMBER;
v_units NUMBER(8,2);
v_price NUMBER(8,2);
v_total NUMBER(10,2);
v_balance NUMBER(10,2);
BEGIN
-- Step 1: Validate buyer
SELECT role, status INTO v_buyer_role, v_buyer_status
FROM USERS WHERE user_id = p_buyer_id;
IF v_buyer_role <> 'CONSUMER' THEN
RAISE_APPLICATION_ERROR(-20103, 'Only consumers can buy energy'); END IF;
IF v_buyer_status <> 'APPROVED' THEN
RAISE_APPLICATION_ERROR(-20104, 'Consumer is not approved'); END IF;
-- Step 2: Lock listing row
SELECT producer_id, units_available, price_per_unit
INTO v_seller_id, v_units, v_price
FROM ENERGY_LISTINGS WHERE listing_id = p_listing_id AND status = 'OPEN' FOR UPDATE;
-- Step 3: Units check
IF p_units > v_units THEN RAISE_APPLICATION_ERROR(-20105,'Not enough units'); END IF;
v_total := p_units * v_price;
-- Step 4-5: Wallet check
SELECT balance INTO v_balance FROM WALLETS WHERE user_id = p_buyer_id FOR UPDATE;
IF v_balance < v_total THEN RAISE_APPLICATION_ERROR(-20106,'Insufficient balance'); END IF;
-- Step 6: Wallet updates
UPDATE WALLETS SET balance = balance - v_total WHERE user_id = p_buyer_id;
UPDATE WALLETS SET balance = balance + v_total WHERE user_id = v_seller_id;
-- Step 7: Update listing
UPDATE ENERGY_LISTINGS SET units_available = units_available - p_units,
status = CASE WHEN units_available - p_units = 0 THEN 'SOLD' ELSE 'OPEN' END
WHERE listing_id = p_listing_id;
-- Step 8: Record transaction + log
INSERT INTO ENERGY_TRANSACTIONS(buyer_id, seller_id, listing_id, units_bought, total_amount)
VALUES(p_buyer_id, v_seller_id, p_listing_id, p_units, v_total)
RETURNING transaction_id INTO p_transaction_id;
INSERT INTO ADMIN_LOGS(action) VALUES('Transaction: ' || p_transaction_id);
END buy_energy;
```

4.4 PKG_REPORT — Reporting & Analytics

All four reporting procedures open a SYS_REFCURSOR that the Node.js gateway iterates and serialises to JSON. This approach keeps SQL logic inside the database and eliminates the risk of SQL injection from application-layer string building.

Procedure	Parameters	Returns (via REF CURSOR)
get_wallet	p_user_id IN NUMBER	wallet_id, user_id, name, balance
get_transactions	p_user_id IN NUMBER	All transactions where buyer or seller = p_user_id, ordered by date DESC
get_pending_users	(none)	All users WHERE status = 'PENDING', ordered by created_at
get_admin_logs	(none)	All ADMIN_LOGS rows, ordered by log_time DESC

5. Backend API Layer (Node.js / Express)

5.1 API Architecture

The backend (server.js) is a minimalist Express application. It defines REST endpoints that each delegate immediately to either **oracleGateway.js** (live Oracle mode) or **demoGateway.js** (in-memory mock mode). The active gateway is selected at startup via the *USE_MOCK_DB* environment variable, enabling frontend development without an Oracle instance.

The Oracle gateway uses the **node-oracledb** library (v6.7.1) to invoke PL/SQL stored procedures with bind variables, avoiding any SQL string concatenation. SYS_REFCURSOR outputs are consumed with *connection.execute()* and the *resultSet* API.

5.2 REST Endpoints

Method	Endpoint	PL/SQL Call	Description
POST	/api/register	PKG_USER.register_user	Create new user account
POST	/api/login	PKG_USER.login_user	Authenticate and return user info
POST	/api/approve	PKG_USER.approve_user	Admin approves / rejects user
POST	/api/listing	PKG_TRADING.add_listing	Producer publishes energy listing
GET	/api/listings	PKG_TRADING.get_listings	Fetch open listings for marketplace
POST	/api/buy	PKG_TRADING.buy_energy	Consumer buys energy units
POST	/api/meter	PKG_TRADING.add_meter_reading	Submit meter reading
GET	/api/wallet/:id	PKG_REPORT.get_wallet	Get wallet balance for user
GET	/api/transactions/:id	PKG_REPORT.get_transactions	Get transaction history for user
GET	/api/pending	PKG_REPORT.get_pending_users	Admin: list pending registrations
GET	/api/logs	PKG_REPORT.get_admin_logs	Admin: list audit log entries

5.3 Oracle Gateway & Demo Gateway

oracleGateway.js — Uses node-oracledb's connection pool to maintain reusable Oracle connections. Every procedure call uses numbered bind variables (*:1, :2, ...*) to safely pass parameters and retrieve OUT values and REF CURSORs. Connection errors propagate back to the Express route handler which returns a 500

JSON error response.

demoGateway.js — Implements the same function signatures as oracleGateway.js using plain JavaScript arrays. This allows the frontend and all API routes to be tested without any database connection, making it ideal for UI demonstrations and CI/CD pipelines. The demo gateway pre-seeds the same three users (producer, consumer, admin) as the seed.sql script.

package.json (v2.0.0): The backend uses ES modules ("type": "module") with three npm scripts: *dev* (node --watch for hot-reload), *start* (production), and *check* (syntax validation without running).

6. Frontend Interface

The frontend is a pure HTML/CSS/JavaScript application that requires no build tools or framework. Three pages cover the complete user journey. A shared *api.js* module abstracts all fetch() calls to the REST API.

6.1 Login & Registration Page (index.html)

The landing page presents two side-by-side cards. The **Login card** includes quick-login buttons for the three demo roles (Producer, Consumer, Admin), each pre-filling the email field. The **Register card** collects name, email, password, role (PRODUCER / CONSUMER) and grid zone. On submission, it calls POST /api/register. A notice explains that new accounts remain PENDING until an admin approves them.

6.2 Dashboard Page (dashboard.html)

After login, users are redirected to the dashboard which adapts its content to the logged-in role:

Role	Dashboard Content
PRODUCER	Wallet balance, "Add Listing" form (units + price + energy type), own active listings, own transaction history, meter reading submission form
CONSUMER	Wallet balance, own transaction history
ADMIN	Pending user approval queue (Approve / Reject buttons), full admin audit log

6.3 Marketplace Page (marketplace.html)

Available only to CONSUMER users. Fetches VW_AVAILABLE_LISTINGS via GET /api/listings and renders each listing as a card showing producer name, zone, units available, price per unit and energy type. A quantity input and "Buy" button trigger POST /api/buy. Success updates the displayed listing's remaining units; error messages from PL/SQL (e.g., "Insufficient wallet balance") are displayed inline.

7. System Constraints & Business Rules

7.1 Core Business Constraints

ID	Constraint	Implementation Detail
BC-01	**Energy Availability**	Units sold cannot exceed available listing units. Enforced in buy_energy (step 3) with SELECT FOR UPDATE to prevent race conditions. Also supported by CHK_LISTING_UNITS (>= 0).
BC-02	**Wallet Sufficiency**	Buyer wallet balance must be >= total purchase cost. Enforced in buy_energy (step 5). CHK_WALLET_BALANCE ensures balance never goes negative.
BC-03	**Role Segregation**	Only PRODUCERs may create listings (error -20101). Only CONSUMERs may buy energy (error -20103). ADMINs have no trading rights.
BC-04	**Approval Gate**	Only APPROVED users may trade. New users start as PENDING. Errors -20102 / -20104 are raised for unapproved participants.
BC-05	**Price Positivity**	price_per_unit must be > 0 (CHK_LISTING_PRICE). total_amount must be > 0 (CHK_TX_TOTAL).
BC-06	**No Negative Units**	units_available >= 0 (CHK_LISTING_UNITS). units_bought > 0 (CHK_TX_UNITS).
BC-07	**Unique Identity**	One email per user (UNIQUE constraint on USERS.email). One wallet per user (UNIQUE FK on WALLETS.user_id).
BC-08	**Referential Integrity**	All FKs are enforced at the DB level, preventing orphan transactions or listings.
BC-09	**Excess Units**	excess_units is always >= 0, enforced by GREATEST() in TRG_METER_EXCESS.
BC-10	**Audit Completeness**	Every state-changing operation (register, approve, list, buy) writes to ADMIN_LOGS.

7.2 Operational Rules

These rules govern the runtime behaviour of the system, independently of the static schema constraints:

- When buy_energy executes, all six sub-operations (validate, lock, check, debit, credit, record) succeed or fail together — Oracle's implicit transaction ensures atomicity.
- A listing transitions from OPEN → SOLD automatically when its last unit is purchased, without any application-level intervention.
- The wallet of each new user is initialised to ■1,000 by the database trigger — application code cannot skip this step.
- Meter readings are immutable once inserted; excess units are computed at write time and stored, not recomputed on read.
- Admin approval is a one-way gate: a REJECTED user must be manually re-approved; the system provides no self-service re-application flow.

- All listing prices are set by the PRODUCER at listing creation time. There is no automated dynamic re-pricing in the current implementation (dynamic pricing was a conceptual feature in Iteration 1; Iteration 2 uses producer-set prices).

7.3 Constraint Enforcement Mechanisms — Summary

Mechanism	Scope	Enforces
PRIMARY KEY	All tables	Unique row identity; prevents duplicate PKs
FOREIGN KEY	Child tables	Referential integrity; no orphan records
UNIQUE	USERS, WALLETS	No duplicate emails; one wallet per user
CHECK	Multiple tables	Domain validation: roles, statuses, prices, units
NOT NULL	Critical columns	Mandatory field enforcement
BEFORE INSERT trigger	METER_READINGS	Auto-compute excess_units
AFTER INSERT trigger	USERS	Auto-create wallet with seed balance
PL/SQL procedure logic	buy_energy, add_listing	Business rule enforcement: role, status, balance, units
SELECT FOR UPDATE	buy_energy	Concurrency control: prevents overselling and overdraft
RAISE_APPLICATION_ERROR	All packages	Structured error propagation to API and frontend

8. Sample Queries & Analytical Capabilities

8.1 Demo Queries (demo_queries.sql)

Five core queries are provided for viva demonstration, each showcasing a different DBMS concept:

Query 1 — Available Listings (View usage + JOIN)

```
SELECT * FROM VW_AVAILABLE_LISTINGS;
-- Internally executes:
-- SELECT l.listing_id, l.producer_id, u.name AS producer_name,
-- u.zone_name, l.units_available, l.price_per_unit, ...
-- FROM ENERGY_LISTINGS l JOIN USERS u ON u.user_id = l.producer_id
-- WHERE l.status = 'OPEN' AND l.units_available > 0 AND u.status = 'APPROVED'
```

Query 2 — Wallet Balances (JOIN)

```
SELECT u.name, u.role, w.balance
FROM USERS u JOIN WALLETS w ON w.user_id = u.user_id;
```

Query 3 — Transaction Details (Multi-table JOIN via View)

```
SELECT * FROM VW_TRANSACTION_DETAILS;
-- buyer.name AS buyer_name, seller.name AS seller_name,
-- units_bought, total_amount, transaction_date
```

Query 4 — Meter Readings with Trigger-Computed Excess

```
SELECT user_id, units_generated, units_consumed, excess_units, reading_time
FROM METER_READINGS;
```

Query 5 — Admin Audit Log

```
SELECT log_id, action, log_time FROM ADMIN_LOGS ORDER BY log_time DESC;
```

8.2 Advanced Analytical Queries

Top Producers by Revenue (GROUP BY + ORDER BY + Aggregate)

```
SELECT u.name AS producer_name,
SUM(t.total_amount) AS total_revenue,
SUM(t.units_bought) AS total_units_sold,
COUNT(*) AS num_transactions
FROM ENERGY_TRANSACTIONS t
JOIN USERS u ON u.user_id = t.seller_id
GROUP BY u.name
ORDER BY total_revenue DESC;
```

Consumers Ranked by Spending (Subquery + RANK)

```
SELECT name, total_spent,
RANK() OVER (ORDER BY total_spent DESC) AS spend_rank
FROM (
SELECT u.name, SUM(t.total_amount) AS total_spent
FROM ENERGY_TRANSACTIONS t
JOIN USERS u ON u.user_id = t.buyer_id
GROUP BY u.name
) ORDER BY spend_rank;
```

Daily Trading Volume (GROUP BY date)

```
SELECT TRUNC(transaction_date) AS trade_date,
COUNT(*) AS num_trades,
SUM(units_bought) AS total_units,
SUM(total_amount) AS daily_revenue
FROM ENERGY_TRANSACTIONS
GROUP BY TRUNC(transaction_date)
ORDER BY trade_date;
```

Listings with Remaining Capacity (HAVING)

```
SELECT l.listing_id, u.name AS producer, l.units_available, l.price_per_unit
FROM ENERGY_LISTINGS l JOIN USERS u ON u.user_id = l.producer_id
WHERE l.status = 'OPEN' AND l.units_available > 10
ORDER BY l.price_per_unit;
```

Producers with No Sales (LEFT JOIN + IS NULL)

```
SELECT u.name AS inactive_producer
FROM USERS u
WHERE u.role = 'PRODUCER'
AND NOT EXISTS (
SELECT 1 FROM ENERGY_TRANSACTIONS t WHERE t.seller_id = u.user_id
);
```

Average Price per Zone (JOIN + GROUP BY)

```
SELECT u.zone_name,
AVG(l.price_per_unit) AS avg_price,
COUNT(l.listing_id) AS num_listings
FROM ENERGY_LISTINGS l
JOIN USERS u ON u.user_id = l.producer_id
GROUP BY u.zone_name
ORDER BY avg_price;
```

8.3 Analytical Insights

Analytical Dimension	Method	Business Insight
Producer Performance	SUM(total_amount) GROUP BY seller_id	Identifies highest-revenue producers; informs incentive design
Consumer Demand	SUM(units_bought) GROUP BY buyer_id	Reveals high-demand consumers who may need priority supply
Price Distribution	AVG/MIN/MAX(price_per_unit)	Tracks price competitiveness across zones
Listing Utilisation	(units_available_original - current) / original	Measures how quickly listings sell out
Wallet Health	MIN(balance) across WALLETS	Early warning for consumers near zero balance
Meter Surplus Trend	AVG(excess_units) over time	Forecasts future listing supply from historical generation data

9. Testing & Validation

9.1 Functional Testing

All key user flows were tested end-to-end using the demo gateway (no Oracle required) and verified against Oracle using SQL*Plus. The table below summarises each test case, the expected outcome, and the actual result.

TC#	Test Case	Expected Outcome	Result
TC-01	Register new PRODUCER user	User created, status=PENDING, wallet auto-created with ■1000	PASS
TC-02	Register duplicate email	Error -20001: Email already exists	PASS
TC-03	Admin approves TC-01 user	status changed to APPROVED; ADMIN_LOGS entry created	PASS
TC-04	Approved PRODUCER adds listing (50 units @ ■6.50)	Listing created, status=OPEN	PASS
TC-05	CONSUMER buys 20 units from TC-04 listing	Buyer wallet -■130, seller wallet +■130, listing units=30, transaction recorded	PASS
TC-06	CONSUMER buys 35 units (30 remaining)	-20105: Not enough units available	PASS
TC-07	CONSUMER with ■0 balance attempts purchase	-20106: Insufficient wallet balance	PASS
TC-08	CONSUMER buys all remaining 30 units	listing status auto-set to SOLD	PASS
TC-09	Submit meter reading (generated=30, consumed=12)	excess_units=18 (computed by trigger)	PASS
TC-10	PRODUCER tries to buy energy	-20103: Only consumers can buy energy	PASS
TC-11	CONSUMER tries to add listing	-20101: Only producers can add listings	PASS
TC-12	Unapproved PRODUCER tries to list	-20102: Producer is not approved	PASS
TC-13	VW_AVAILABLE_LISTINGS reflects TC-08 (SOLD)	SOLD listing not returned in view	PASS
TC-14	Admin logs populated throughout	All events logged with timestamps	PASS

9.2 Constraint Validation

CHECK constraints were validated by attempting direct SQL INSERTs that violate domain rules:

Constraint	Violation Attempted	Oracle Response
chk_user_role	INSERT INTO USERS ... role = 'HACKER'	ORA-02290: check constraint violated
chk_user_status	UPDATE USERS SET status = 'BANNED'	ORA-02290: check constraint violated
chk_wallet_balance	UPDATE WALLETS SET balance = -50	ORA-02290: check constraint violated
chk_listing_price	INSERT INTO ENERGY_LISTINGS ... price = 0	ORA-02290: check constraint violated
chk_tx_units	INSERT INTO ENERGY_TRANSACTIONS ... units = -5	ORA-02290: check constraint violated
fk_wallet_user	DELETE FROM USERS WHERE user_id = 1 (has wallet)	ORA-02292: integrity constraint violated

9.3 Error Handling

The system implements a two-level error handling strategy:

- **PL/SQL level:** RAISE_APPLICATION_ERROR with specific codes (-20001 through -20106) and descriptive messages. These propagate as Oracle exceptions back to the Node.js driver.
- **Node.js API level:** All gateway calls are wrapped in try/catch. Oracle exceptions are caught and returned as HTTP 400/500 responses with a JSON body containing the error message. The frontend displays these messages inline without crashing the page.
- **Demo mode:** demoGateway.js throws JavaScript Error objects with the same message strings, so the frontend error display path is identical regardless of the backend mode.

10. Conclusion & Future Work

10.1 Project Achievements

This project successfully delivered a working, full-stack, database-driven peer-to-peer solar energy trading platform. The key achievements are:

Achievement	Detail
Normalised Relational Schema	6-table schema satisfying 3NF, implemented in Oracle with IDENTITY PKs, FKs, CHECK constraints, UNIQUE constraints and NOT NULL constraints.
PL/SQL Business Logic	3 packages (PKG_USER, PKG_TRADING, PKG_REPORT) encapsulating 11 procedures, all with structured error handling.
Atomic Transaction Processing	buy_energy uses SELECT FOR UPDATE to provide serialisable isolation, preventing overselling and overdraft in concurrent scenarios.
Automatic Database Behaviours	2 triggers (TRG_CREATE_WALLET, TRG_METER_EXCESS) enforce invariants that application code cannot bypass.
RESTful API Gateway	11 HTTP endpoints mapping cleanly to PL/SQL procedures via node-oracledb.
Role-Aware Frontend	3 HTML pages with role-conditional rendering for PRODUCER, CONSUMER and ADMIN.
Demo Mode	Complete in-memory fallback enabling UI development and presentation without Oracle.
Audit Trail	ADMIN_LOGS provides a full, immutable record of all system events.
Comprehensive Testing	14 test cases covering happy paths, error paths and constraint violations.

10.2 Limitations

- Passwords are stored in plaintext (SHA-256 hashing would be required for production).
- No session management or JWT tokens; the frontend stores the user object in sessionStorage.
- Dynamic pricing (demand/supply formula from Iteration 1) was replaced by producer-set prices in Iteration 2 for implementation clarity.
- The system currently supports a single energy type (SOLAR) and a single currency (INR).
- Concurrent stress testing beyond the demo dataset has not been conducted.
- There is no real-time push notification for buyers when a new listing is posted.

10.3 Future Enhancements

Enhancement	Description

Dynamic Pricing Engine	Re-introduce the demand-factor / supply-factor formula from Iteration 1 as a scheduled Oracle job that recomputes listing prices every hour based on current supply and demand ratios.
Smart Meter Integration	Connect to IoT smart meter APIs (MQTT / REST) to auto-populate METER_READINGS without manual submission.
Blockchain Settlement	Record each energy_transaction hash on a public blockchain (Ethereum / Hyperledger) for immutable, cross-party verification.
Mobile Application	React Native app consuming the same REST API, with push notifications for new listings and completed transactions.
DISCOM Integration	API integration with Distribution Companies for net-metering credits when excess units are exported to the main grid.
Advanced Analytics Dashboard	Oracle APEX or a React dashboard exposing time-series charts, zone-level heatmaps and price forecasting.
Multi-Currency / Token Support	Introduce an energy token (e.g., 1 token = 1 kWh) alongside INR, enabling cross-border energy trading in a regulatory sandbox.
Password Hashing & OAuth2	Replace plaintext passwords with bcrypt hashing and add Google / Aadhaar OAuth2 login.

11. References

Textbooks & Standards

[1] Ramakrishnan, R. & Gehrke, J. (2003). *Database Management Systems*, 3rd Edition. McGraw-Hill.

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts*, 7th Edition. McGraw-Hill.

[3] Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz*, 2nd Edition. O'Reilly Media.

Oracle Official Documentation

[1] Oracle Corporation. (2023). *Oracle Database 19c PL/SQL Language Reference*. docs.oracle.com.

[2] Oracle Corporation. (2023). *Oracle Database 19c SQL Language Reference*. docs.oracle.com.

[3] Oracle Corporation. (2023). *Oracle Database 19c Concepts Guide*. docs.oracle.com.

[4] Oracle Corporation. (2024). *Oracle Database Free (23ai) Documentation*. docs.oracle.com.

Node.js & API References

[1] Oracle Corporation. (2024). *node-oracledb 6.x Documentation*. oracle.github.io/node-oracledb.

[2] Express.js. (2024). *Express 4.x API Reference*. expressjs.com.

[3] Node.js Foundation. (2024). *Node.js 20 Documentation*. nodejs.org.

Energy Domain & Research

[1] Ministry of New and Renewable Energy, Government of India. (2024). *Annual Report 2023–24: Rooftop Solar Programme*. mnre.gov.in.

[2] International Energy Agency. (2024). *Renewables 2024: Analysis and Forecast to 2030*. IEA Publications.

[3] Zhang, C., Wu, J., Zhou, Y., Cheng, M., & Long, C. (2018). Peer-to-peer energy trading in a microgrid. *Applied Energy*, 220, 1–12.

[4] Tushar, W., Saha, T. K., Yuen, C., Smith, D., & Poor, H. V. (2020). Peer-to-peer trading in electricity networks: An overview. *IEEE Transactions on Smart Grid*, 11(4), 3185–3200.

Web & Development Resources

[1] MDN Web Docs. (2024). *HTML5, CSS3 and JavaScript Reference*. developer.mozilla.org.

[2] W3Schools. (2024). *SQL Tutorial and Reference*. w3schools.com.

[3] GeeksforGeeks. (2024). *PL/SQL Triggers, Packages and Cursors*. geeksforgeeks.org.

ACKNOWLEDGEMENT

We sincerely thank **Dr. Simran** for her guidance and constructive feedback throughout the UCS310 course. We also acknowledge Thapar Institute of Engineering and Technology for providing access to Oracle Database resources and a structured curriculum that made this project possible. This project represents our practical understanding of relational database design, PL/SQL programming, and full-stack application development applied to a real-world renewable energy domain.

Tarun Mahindra (1024030177) | Parth Puri (1024030178)